

Zahlen schreiben und Bits lesen

Wozu? Ein Register speichert Bits. Du brauchst eine Bitbreite und eine Interpretation, um daraus eine Zahl zu lesen. Binär und Hex sind Schreibweisen; unsigned und Zweierkomplement bestimmen die Bedeutung des Bitmusters.

Stellenwerte: Nur die Stellen mit einer 1 mitzählen

Position	7	6	5	4	3	2	1	0
Unsigned	128	64	32	16	8	4	2	1
Beispiel	0	1	0	1	1	0	1	1

$$0101\,1011_2 = 64 + 16 + 8 + 2 + 1 = 91_{10}$$

Die rechte Stelle zählt $2^0 = 1$. Jede Stelle nach links zählt doppelt so viel. Dezimal funktioniert genauso, nur mit 1, 10, 100, ... statt 1, 2, 4, ...

Dezimal nach Binär

Zerlege die Zahl in Zweierpotenzen: $45 = 32 + 8 + 4 + 1$. Mit den Stellen 32, 16, 8, 4, 2, 1 ergibt das 101101_2 ; auf acht Bits aufgefüllt: 0010 1101. Alternativ: wiederholt durch 2 teilen, Reste von unten nach oben lesen.

Hex: Eine Ziffer entspricht vier Bits

Hex	Dez.	Bits	Hex	Dez.	Bits	Hex	Dez.	Bits	Hex	Dez.	Bits
0	0	0000	4	4	0100	8	8	1000	C	12	1100
1	1	0001	5	5	0101	9	9	1001	D	13	1101
2	2	0010	6	6	0110	A	10	1010	E	14	1110
3	3	0011	7	7	0111	B	11	1011	F	15	1111

Von rechts in Vierergruppen teilen; eine unvollständige Gruppe links mit Nullen ergänzen.

$$0xB7 = 1011\,0111_2 = 11 \cdot 16 + 7 = 183_{10}$$

Achtung: 0x ist das Hexpräfix, 0b das Binärpräfix. 0x23 ist $2 \cdot 16 + 3 = 35$, nicht 23 dezimal. Hex legt kein Vorzeichen fest.

Breite und Einheiten

N Bits haben 2^N Muster. Unsigned reichen sie von 0 bis $2^N - 1$; die Null zählt als eine Möglichkeit mit.

1 Nibble	4 Bits = 1 Hexziffer	1 Byte	8 Bits = 2 Hexziffern
LSB	niederwertigstes Bit rechts	MSB	höchstwertigstes Bit links
1 KiB	$1024 = 2^{10}$ Byte	1 kB	1000 Byte

Ein Word ist eine architekturabhängige Dateneinheit. 1 MiB = 2^{20} Byte, 1 GiB = 2^{30} Byte. Kleinbuchstabe b steht häufig für Bit, B für Byte.

02 / INTERPRETATION

Negative Zahlen und das Warum

Warum Zweierkomplement?

Wir wollen dieselbe Binäraddition auch für negative Zahlen verwenden. Mit vier Bits wirkt das Muster 1111 beim Addieren wie „eins abziehen“:

$$\underbrace{0101_2}_5 + \underbrace{1111_2}_{\text{Muster für } -1} = 10100_2 \longrightarrow 0100_2 = 4.$$

Nur die rechten vier Bits werden gespeichert. Unsigned wäre dies $5 + 15 = 20$; nach Wegfall der zusätzlichen 16 bleibt 4. Deshalb ist 15 als Bitmuster praktisch für -1 . Allgemein repräsentiert $2^N - k$ die Zahl $-k$, wenn sie im signed Bereich liegt.

Die linke Stelle zählt negativ

Interpretation	Stellenwerte bei vier Bits			Muster 1101
Unsigned	+8	4	2	1
Zweierkomplement	-8	4	2	1

Kein bloßes Minuszeichen: Im Zweierkomplement sind die restlichen Bits nicht der Betrag. 1101 bedeutet -3 , nicht -5 . Das System „Vorzeichen und Betrag“ würde -5 ergeben und hat zwei Nullen; es ist nicht das System unserer Funktionen.

Die Bitbreite gehört zur Bedeutung

Muster	Bits	Unsigned	Zweierkomplement
1101	4	13	-3
0000 1101	8	13	+13
1111 1101	8	253	-3

Mit N Bits ist der höchste Stellenwert -2^{N-1} . Daraus folgen:

$$\boxed{\text{unsigned: } 0 \dots 2^N - 1} \quad \boxed{\text{signed: } -2^{N-1} \dots 2^{N-1} - 1}$$

Bits	Unsigned	Zweierkomplement
1	0 bis 1	-1 bis 0
4	0 bis 15	-8 bis 7
5	0 bis 31	-16 bis 15
8	0 bis 255	-128 bis 127

Negieren und Verbreitern

Vorzeichen umkehren: Bits invertieren, 1 addieren, N Bits behalten.

$$+3 : 0011 \longrightarrow 1100 \longrightarrow 1101 : -3$$

Ausnahme: Die kleinste negative Zahl hat bei gleicher Breite kein positives Gegenstück. Vier-Bit- -8 lässt sich nicht als Vier-Bit- $+8$ negieren.

Vorzeichenerweiterung: Links das bisherige Vorzeichenbit wiederholen. 1101 wird 1111 1101 (-3 bleibt -3). Positive Zahlen bekommen links Nullen. Unsigned Zahlen werden mit Nullen erweitert.

Addition, Carry und signed Overflow

Binäre Addition und Subtraktion

$0 + 0 = 0$, $0 + 1 = 1$, $1 + 1 = 10_2$. Bei $1 + 1$ schreiben wir 0 und tragen 1 nach links. Mit eingehendem Übertrag: $1 + 1 + 1 = 11_2$; 1 schreiben und 1 weitertragen.

Subtraktion wird Addition der negierten zweiten Zahl: $a - b = a + (-b)$.

$$5 - 3: \quad 0101 + 1101 = 10010 \quad \longrightarrow \quad 0010 = 2.$$

Zwei unterschiedliche Fragen

Carry-out: Wird bei Addition der N -Bit-Muster eine 1 aus der höchsten Stelle herausgetragen? Bei unsigned Addition zeigt dies einen Überlauf an. Ein Übertrag *innerhalb* der Rechnung ist noch kein Carry-out.

Signed Overflow: Liegt die mathematisch richtige Summe außerhalb des signed Zahlenbereichs? Zwei positive Operanden können dann ein negatives Ergebnis liefern; zwei negative ein nichtnegatives. Unterschiedliche Vorzeichen führen bei gültigen Operanden nicht zu signed Overflow.

Signed Rechnung	Gespeichert	Wert	Carry-out	Signed Overflow
$-1 + 1$	0000	0	Ja	Nein
$7 + 1$	1000	-8	Nein	Ja
$-8 + (-1)$	0111	7	Ja	Ja
$2 + (-5)$	1101	-3	Nein	Nein

Alle Beispiele verwenden vier Bits. Die **richtige Summe** und der **gespeicherte signed Wert** sind bei Overflow verschieden.

Was die Python-Funktion nachbildet

Python-Integer sind nicht von selbst auf unsere Registerbreite begrenzt. Für die mathematische Summe $s = a + b$ gilt:

$$\begin{aligned} \text{overflow} &= (s < -2^{N-1}) \vee (s > 2^{N-1} - 1), \\ \text{raw} &= s \bmod 2^N, \\ \text{ergebnis} &= \text{from_twos_complement}(\text{raw}, N). \end{aligned}$$

raw beschreibt das gespeicherte Muster als unsigned Zahl. Erst die letzte Zeile interpretiert es als signed Zahl.

Warum Modulo (%)?

Bei vier Bits entspricht „die vier rechten Bits behalten“ dem Rest modulo 16:

Summe	Rechnung	raw	Signed Interpretation
18	$18 = 1 \cdot 16 + 2$	2	$0010 \rightarrow 2$
8	$8 = 0 \cdot 16 + 8$	8	$1000 \rightarrow -8$
-3	$-3 = (-1) \cdot 16 + 13$	13	$1101 \rightarrow -3$
-9	$-9 = (-1) \cdot 16 + 7$	7	$0111 \rightarrow 7$

Bei positivem Divisor 16 liefert Python mit % 16 immer einen Rest von 0 bis 15. Auch bei negativen Summen entsteht so das richtige gespeicherte Muster.

Ein vollständiger Ablauf

`add_with_overflow_flag(7, 1, 4)`: Summe 8; Bereich $-8 \dots 7$; Overflow wahr; $8 \bmod 16 = 8$; Muster 1000; signed Wert -8; Rückgabe $(-8, \text{True})$.

Von der Zahl zum formatierten Text

Keine Imports nötig. bin, hex, print, int, str und ValueError sind eingebaut. Schlüsselwörter sind im Code fett, Kommentare kursiv; die Darstellung bleibt vollständig schwarz-weiß.

Zahl und Text unterscheiden

```
value = 6           # int: eine Zahl
text = bin(value)   # str: "0b110"
print(6 + 1)        # 7
print("6" + "1")    # 61: Texte werden aneinandergesetzt
```

bin() und hex() liefern Text. Die Präfixe 0b und 0x kennzeichnen die Basis. Sie sind keine zusätzlichen Ziffern des Zahlenwerts.

Die Umwandlung in einzelnen Schritten

```
mit_praefix = bin(9)           # "0b1001"
ohne_praefix = mit_praefix[2:] # "1001"
ergebnis = ohne_praefix.zfill(8) # "00001001"

mit_praefix = hex(15)          # "0xf", kleines f
```

Indexierung: Im Text 0b1001 haben die Zeichen die Positionen 0, 1, 2, 3, 4, 5. [2:] übernimmt ab Position 2 bis zum Ende. Es entfernt das Präfix und rechnet den Text nicht in Dezimal um.

```
ohne_praefix = mit_praefix[2:] # "f"
gross = ohne_praefix.upper()   # "F"
ergebnis = gross.zfill(2)      # "0F", nicht "FF"
```

zfill(n) ergänzt links Nullen bis zur **Gesamtlänge** n. Ä" .zfill(3) ergibt "00A". Es kürzt zu lange Texte nicht und fügt kein Präfix hinzu. Deshalb prüfen wir den Wertebereich separat.

Wie viele Hexstellen? Erst addieren, dann teilen

$$\text{Hexstellen} = \left\lceil \frac{N}{4} \right\rceil \quad \text{Python: (bits + 3) // 4}$$

// zählt bei positiven Zahlen die vollständigen Gruppen: $19 = 4 \cdot 4 + 3$, also $19 // 4 == 4$. Der Rest wird nicht aufgerundet. Das vorherige +3 sorgt dafür, dass jeder ursprüngliche Rest von 1, 2 oder 3 noch eine Gruppe mitzählen lässt; bei Rest 0 entsteht keine zusätzliche Gruppe.

Bits	Vierergruppen	Formel	Hexstellen
8	2 volle	$11 // 4$	2
9	2 volle, 1 Bit übrig	$12 // 4$	3
12	3 volle	$15 // 4$	3
16	4 volle	$19 // 4$	4

Klammern! $(8 + 3) // 4$ ergibt 2; $8 + 3 // 4$ ergibt 8, weil zuerst dividiert wird.

Funktionen: = speichert einen Wert, == vergleicht. return gibt zurück und beendet den Aufruf; print zeigt etwas an. return gehört in eine Funktion. Einrückung bestimmt die Zugehörigkeit. Typhinweise wie : int oder -> str erzwingen keine Typprüfung.

05 / REFERENZCODE

Die ersten drei Funktionen

Für alle Eingaben setzen wir ganze Zahlen voraus. bits muss mindestens 1 sein; ein unsigned Muster muss zwischen 0 und $2^{\text{bits}} - 1$ liegen.

```
def to_bin(value: int, bits: int) -> str:
    if bits < 1:
        raise ValueError("Bitbreite muss mindestens 1 sein.")
    if value < 0 or value > 2**bits - 1:
        raise ValueError("Zahl passt nicht in die Bitbreite.")

    mit_praefix = bin(value)
    ohne_praefix = mit_praefix[2:]
    ergebnis = ohne_praefix.zfill(bits)
    return ergebnis

def to_hex(value: int, bits: int) -> str:
    if bits < 1:
        raise ValueError("Bitbreite muss mindestens 1 sein.")
    if value < 0 or value > 2**bits - 1:
        raise ValueError("Zahl passt nicht in die Bitbreite.")

    mit_praefix = hex(value)
    ohne_praefix = mit_praefix[2:]
    gross = ohne_praefix.upper()
    hex_stellen = (bits + 3) // 4
    return gross.zfill(hex_stellen)

def from_twos_complement(raw: int, bits: int) -> int:
    if bits < 1:
        raise ValueError("Bitbreite muss mindestens 1 sein.")
    if raw < 0 or raw > 2**bits - 1:
        raise ValueError("Bitmuster passt nicht in die Bitbreite.")

    grenze = 2**(bits - 1)
    if raw < grenze:
        return raw
    return raw - 2**bits
```

Warum wird beim Dekodieren 2^N abgezogen?

Das höchste Bit wechselt seinen Stellenwert von $+2^{N-1}$ zu -2^{N-1} . Die Differenz ist -2^N . Alle anderen Stellen bleiben unverändert.

$$\text{signed}(r, N) = \begin{cases} r, & r < 2^{N-1}, \\ r - 2^N, & r \geq 2^{N-1}. \end{cases}$$

Genau an der Grenze beginnt der negative Bereich: <, nicht <=. Bei vier Bits muss raw=8 zu -8 werden.

```
assert to_bin(6, 4) == "0110"
assert to_hex(15, 8) == "0F"
assert from_twos_complement(13, 4) == -3
assert from_twos_complement(13, 8) == 13
```

06 / REFERENZCODE UND PRÜFUNG

Addition und dein Tagesabschluss

```
def add_with_overflow_flag(a: int, b: int,
                           bits: int) -> tuple[int, bool]:
    if bits < 1:
        raise ValueError("Bitbreite muss mindestens 1 sein.")

    minimum = -(2**(bits - 1))
    maximum = 2**(bits - 1) - 1
    if a < minimum or a > maximum:
        raise ValueError("a passt nicht in die Bitbreite.")
    if b < minimum or b > maximum:
        raise ValueError("b passt nicht in die Bitbreite.")

    summe = a + b
    overflow = summe < minimum or summe > maximum
    raw = summe % (2**bits)
    ergebnis = from_twos_complement(raw, bits)
    return ergebnis, overflow

assert add_with_overflow_flag(2, 3, 4) == (5, False)
assert add_with_overflow_flag(2, -5, 4) == (-3, False)
assert add_with_overflow_flag(7, 1, 4) == (-8, True)
assert add_with_overflow_flag(-8, -1, 4) == (7, True)
```

Die Rückgabe ist ein **Tupel**: gespeicherte signed Zahl und ein Wahrheitswert. True bedeutet „Overflow“, False bedeutet „kein Overflow“. Ungültige *Operanden* lösen ValueError aus; ein Überlauf der *Summe gültiger Operanden* wird mit dem Flag gemeldet.

Im Terminal ausprobieren

Aktiviere die Umgebung und wechsle in den Übungsordner. Die folgenden Befehle sind Shell-Code, kein Python:

```
source "$HOME/.venvs/ai-hardware-day04/bin/activate"
cd "$HOME/Learning/Semiconductors : AI Hardware/ai-hardware-journey/01_digital/01_zahlensysteme/tag04"
python
```

In der Python-Eingabe kannst du dann schreiben:

```
from hardware_numbers.numbers_lernversion import from_twos_complement
print(from_twos_complement(19, 5)) # -13
exit()
```

Tests: Im Übungsordner startet `python -m pytest -q` die Tests. Sie prüfen die ursprüngliche Datei `numbers.py`, nicht die Lernversion. Fertige Funktionen dafür in die ursprüngliche Datei übertragen. Lose Experimente mit undefinierten Variablen dort entfernen.

Was du an Tag 4 beherrschen solltest

Binär/Hex/Dezimal umrechnen; unsigned und signed Bereiche bestimmen; Zweierkomplement erklären und dekodieren; addieren, subtrahieren und Carry von signed Overflow unterscheiden; Vorzeichen korrekt erweitern. Vier Funktionen implementieren und mindestens 20 Tests inklusive Randfällen bestehen lassen. Prüfe besonders 0, -2^{N-1} , $2^{N-1} - 1$, 1 Bit und ungültige Eingaben.

Grundlage: Harris & Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Abschnitt 1.4, Buchseiten 7–17; dein Lernplan Woche 1, Tag 4. Eigene Erklärungen und Python-Referenz für unsere Übung.